

PyQCU 分布式 Strict MultiGrid: 双 P100 illegal-memory debug、并行优化与全矩阵 QUDA 对照

PyQCU 工程分析记录

2026 年 9 月 21 日

摘要

本文记录 PyQCU 分布式 Strict-MG 从双 P100 旧版 clover-gauge MPI halo 非法访问修复, 到真正 rank-sliced HDF5 读取、粗层通信优化, 再到 $2 \times 2 \times 3 \times 2 \times 3 = 72$ 个公平对照单元 (共 144 个 side case) 的完整测试。最终矩阵覆盖 $c64/c128$ 、 $8^3 \times 16$ 、 16^4 、 $16 \times 32 \times 32 \times 48$ ($c64$) 与 $16 \times 16 \times 32 \times 32$ ($c128$)、cold + 2 warmup + 5 steady、trace on/off、BiCGstab/MG-2/MG-3、V100 单卡与双 P100。所有 144 个 side case 均通过真残差门禁, 72 个比较均 `fair=true`。MG-2/3 的 48 个比较中, PyQCU 在 36 个单元更快, QUDA/PyQCU 比值中位数 1.298; V100 的 MG-2/3 中位数 2.492, 双 P100 的中位数 1.063。BiCGstab reference 的 24 个比较全部慢于 QUDA, 中位数 0.243, 因此报告中严格分列, 不用 MG 结果掩盖 reference 路径的不足。本文同时记录两个失败的粗层归约实验 (triple-dot 与 pipelined BiCGStab) 及其回退, 避免把数值失稳路径写成性能优化。

目录

1 范围、验收与证据等级	1
2 双 P100 非法访问的根因与修复	2
2.1 设备绑定根因	2
2.2 pack kernel 边界根因	2
3 真正的分布式输入与粗层通信优化	2
3.1 rank-sliced HDF5	2
3.2 Strict-MG 数据路径优化	3
4 QUDA 的优势与 PyQCU 的剩余短板	3
4.1 QUDA 的结构性优势	3
4.2 PyQCU 的剩余短板	3
4.3 两条失败优化路径	4
5 测试协议与覆盖	4

1 范围、验收与证据等级 2

6 性能结果 4

6.1 总览 4

6.2 逐层耗时 5

6.3 残差 5

7 正确性与工程验证 9

8 遗留问题与适用边界 9

9 复现与产物 9

10 结论 10

11 2026-09-27 后续状态 10

1 范围、验收与证据等级

表 1: 最终验收结果

项目	实测结果
笛卡尔 side case	144/144 个单元存在且两侧 status=ok
公平比较	72/72 个 fair=true, 无 config hash、input bundle 或残差门禁失败
相位与层级记录	432 个 phase cell、336 个 MG level cell 全覆盖
真残差	PyQCU 最大 8.01×10^{-7} ; QUDA 最大 7.97×10^{-7} , 全部低于各自 gate
分布式输入	双 rank 下 gauge/source/null 均采用 rank-sliced HDF5 hyperslab; round-trip 逐元素误差 0
旧版 CUDA illegal access	原失败路径不再复现; 随后完成全部双 P100 长测
性能结论	MG-2/3 36/48 胜; V100 中位数 2.492x, 双 P100 中位数 1.063x
BiCGstab reference	0/24 胜, 中位数 0.243x, 单独列为短板

证据等级如下:

- E1: 本机 CUDA/MPI 实测、JSON/trace/validate 输出与 CUDA sanitizer;
- E2: QUDA 1.1.0 与 PyQCU 源码逐层对照、算法推导;
- E3: 仅用于历史背景的旧报告结论。

最终机器可读产物为 data/mg_matrix_final_combined/units/、logs/data/mg_matrix_final_analysis_full.json、logs/data/mg_matrix_final_validation_full.json、data/mg_matrix_final_report/mg_matrix.csv 与 data/mg_matrix_final_report/mg_levels.csv。

2 双 P100 非法访问的根因与修复

2.1 设备绑定根因

旧版双 P100 收集器在 `CUDA_VISIBLE_DEVICES=0,1` 下由 Python 选择 Torch 逻辑设备 0/1, 而 C++ `LatticeSet` 仍强制 `cudaSetDevice(0)`。rank 1 因此把 GPU1 张量指针交给 GPU0 上下文, 在旧版 `clover-gauge pick_up_u_* pack kernel` 中触发 CUDA illegal memory access。修复分两处:

- `cpp/cuda/qcu/include/define.h:271` 将 `_TEST_SINGLE_GPU_MULTI_RANK_` 设为 0;
- `cpp/cuda/qcu/include/lattice_set.h:272--288` 在多 rank 路径读取 `PYQCU_MPI_DEVICE_ID`, 回退 local rank; `pyqcu/testing/qcu/strict/quda_comparison/bench_strict_vs_quda.py:3181` 把 Torch 实际设备索引写入该环境变量。

2.2 pack kernel 边界根因

旧版 10 个 `pick_up_u_* (x/y/z/t, xy/xz/xt/yz/yt/zt)` pack kernel 在线程数按 face grid 向上取整时没有上界检查; 尾线程越过 face 元素数。 `cpp/cuda/qcu/src/clover_dslash_comm.cu` 与 `cpp/cuda/qcu/include/lattice_clover_dslash.h` 中的所有 1D/2D pack kernel 现在在索引解码前返回越界线程。修复后的原失败命令退出码为 0; compute-sanitizer memcheck 未再报告 illegal access。

3 真正的分布式输入与粗层通信优化

3.1 rank-sliced HDF5

旧 worker 每个 MPI rank 读取完整 `gauge/source/null` HDF5, 因此双 rank 重复占用数 GiB host memory。新实现 `pyqcu/testing/qcu/strict/quda_comparison/bench_strict_vs_quda.py:2520` 的 `_load_h5_local_array` 使用 h5py MPI-IO hyperslab:

- `gauge/source` 的 checkerboard-compressed 数据集按 $x, y, z, t/2$ 读取 rank slab; 局部原点和长度均为偶数, 所以局部 checkerboard 与全局 checkerboard 一致;
- canonical full null-vector 按 x, y, z, t 读取 rank slab;
- 单 rank 保留全量读取语义; 多 rank 对 dtype/shape/file-size 重新校验, 父进程仍在启动前计算完整 logical SHA-256。

双 rank 小格实测: `gauge`、`source`、`null` 三类 slab 与全量读取切片逐元素 `max_abs=0`。

3.2 Strict-MG 数据路径优化

1. 活跃 halo slot staging. `cpp/cuda/qcu/src/apply_multigrid_strict.cu:1690--1760` 仅复制 process grid 中实际活跃的 (dim, side) slot, 而不是无条件复制完整 8-slot 缓冲;

2. **link halo cache.** `cpp/cuda/qcu/src/apply_multigrid_strict.cu:1808--1835` 按 $(parity, dim)$ 缓存 static preconditioned link ghost, pointer 不变时跳过重复 pack/MPI;
3. **prolongation 与加法融合.** `cpp/cuda/qcu/src/apply_multigrid_strict.cu:1489--1531` 用 add 参数在同一 kernel 中完成 prolongation 加 corrected coarse contribution;
4. **trace-on cache 复用.** `pyqcu/testing/qcu/strict/quda_comparison/bench_strict_vs_quda.py:6135` 允许显式 `cache-expect=hit` 时 cold/main 两个 phase 复用同一个 hierarchy cache;
5. **矩阵编排.** `pyqcu/testing/qcu/strict/quda_comparison/bench_mg_matrix_full.py:394` 记录 `strict_cache_expect`, trace-on 优先复用对应 trace-off cache。

4 QUDA 的优势与 PyQCU 的剩余短板

4.1 QUDA 的结构性优势

- QUDA 的 transfer、block orthogonalization、prolongator/restrictor、coarse $X/Y/\hat{Y}$ 与 coarse-PC 均有成熟专用 kernel;
- QUDA 的粗层求解器默认使用 CA-GCR, 一次 block dot 可摊销多个 Krylov 方向的归约延迟;
- QUDA 有完整的 QMP/NVSHMEM/coarse halo、逐层精度和长期硬件回归;
- QUDA 的 coarse dslash 与通信路径可直接复用生产 kernel, 而 PyQCU Strict 的粗层仍是“宿主侧全局归约 + halo 交换”的较薄实现。

4.2 PyQCU 的剩余短板

- 最粗层的每次 BiCGStab 迭代仍需 `host → device → host` 标量回传; 双 P100 上粗层延迟是首要瓶颈;
- 旧版 Strict 的 mixed precision、奇数 coarse extent 和 non-nearest-neighbour stencil 仍有 fail-closed 限制;
- Galerkin setup 与 runtime cache 虽已分离, 但 cache miss 仍是分钟级成本; 矩阵编排已通过 trace-off/trace-on 共享 cache 降低重复;
- BiCGStab reference 没有 MG 的多层预处理, 单次迭代成本高, 在 24 个 L1 对照中全部慢于 QUDA。

4.3 两条失败优化路径

为降低粗层归约延迟, 曾实现并测试:

1. 将 $\langle s, s \rangle, \langle t, s \rangle, \langle t, t \rangle$ 合并成一个 triple reduction;

2. 将 BiCGStab 的 ρ 延迟到残差更新后, 与 $\langle r, r \rangle$ 合成 dot-pair, 形成 pipelined 版本。

两者在小格、中格能收敛, 但在 $16 \times 32 \times 32 \times 48$ 、c64、双 P100 的 FGMRES 外循环中出现残差停滞; trace 显示外残差长期停在 10^{-3} 量级而 coarse residual 仍约 0.2。这属于当前分布式 BiCGStab 对归约重排的数值敏感性, 不是可接受的性能优化。两路径均已从最终源码移除, 正式结果使用原始、已验证的归约顺序。该失败实验说明“减少同步次数”不能以牺牲粗层递推稳定性为代价。

5 测试协议与覆盖

表 2: 最终矩阵协议

轴	取值
side	PyQCU、QUDA
precision	c64、c128
c64 lattice	$8^3 \times 16$ 、 16^4 、 $16 \times 32 \times 32 \times 48$
c128 lattice	$8^3 \times 16$ 、 16^4 、 $16 \times 16 \times 32 \times 32$
trace	off、on
levels	BiCGstab reference、MG-2、MG-3 (执行顺序 2,3,1)
phases	cold 1、warmup 2、steady 5
solver gate	c64 5×10^{-6} 、c128 5×10^{-8} true residual
device	V100-SXM2-32GB 单卡; P100-PCIE-16GB $\times 2$, process grid [2, 1, 1, 1]
shared parameters	coarse-tol 0.003; P100 全组 nu=1; V100 c64 nu=1; V100 c128 nu=2

矩阵共 144 个 side case。每个 PyQCU/QUDA 对共享同一 canonical null-vector、同一 gauge、同一 solver tolerance、同一 restart 与同一 coarse-tol。两个侧别均记录 phase、iterations、true residual, trace-on 侧额外记录逐层 stage。

6 性能结果

6.1 总览

表 3: MG-2/3 分组结果; 比值为 QUDA/PyQCU, > 1 表示 PyQCU 更快。每组 6 个单元 (3 lattice $\times$ 2 trace)。

device	precision	levels	median	wins	losses	range
P100	c64	MG-2	0.923	1	5	0.645–1.110
P100	c64	MG-3	1.157	4	2	0.640–1.777
P100	c128	MG-2	1.072	4	2	0.823–1.294
P100	c128	MG-3	1.098	4	2	0.661–1.365
V100	c64	MG-2	2.212	5	1	0.999–4.446
V100	c64	MG-3	2.184	6	0	1.207–9.115
V100	c128	MG-2	3.012	6	0	1.405–16.393
V100	c128	MG-3	3.795	6	0	1.602–19.587

表 4: 汇总统计

集合	cases	median	wins	losses	range
MG-2/3 全体	48	1.298	36	12	0.640–19.587
BiCGstab reference	24	0.243	0	24	0.164–0.468
V100 MG-2/3	24	2.492	23	1	1.000–19.587
P100 MG-2/3	24	1.063	13	11	0.640–1.777

代表性单元:

- 双 P100、 $16 \times 32 \times 32 \times 48$ 、c64、MG-2、trace-off: PyQCU 9.204s、QUDA 7.979s, 比值 0.867; 这是双 P100 最明显的大格退化项;
- 同格点 MG-3、trace-off: PyQCU 2.812s、QUDA 4.998s, 比值 1.777;
- V100、 $16 \times 16 \times 32 \times 32$ 、c128、MG-2、trace-off: PyQCU 2.487s、QUDA 4.017s, 比值 1.615;
- 同格点 MG-3: PyQCU 1.418s、QUDA 3.459s, 比值 2.439;
- 全局最大值出现在 V100、 $8^3 \times 16$ 、c128、MG-3、trace-off: 比值 19.587, 但该结论仅适用于该固定协议。

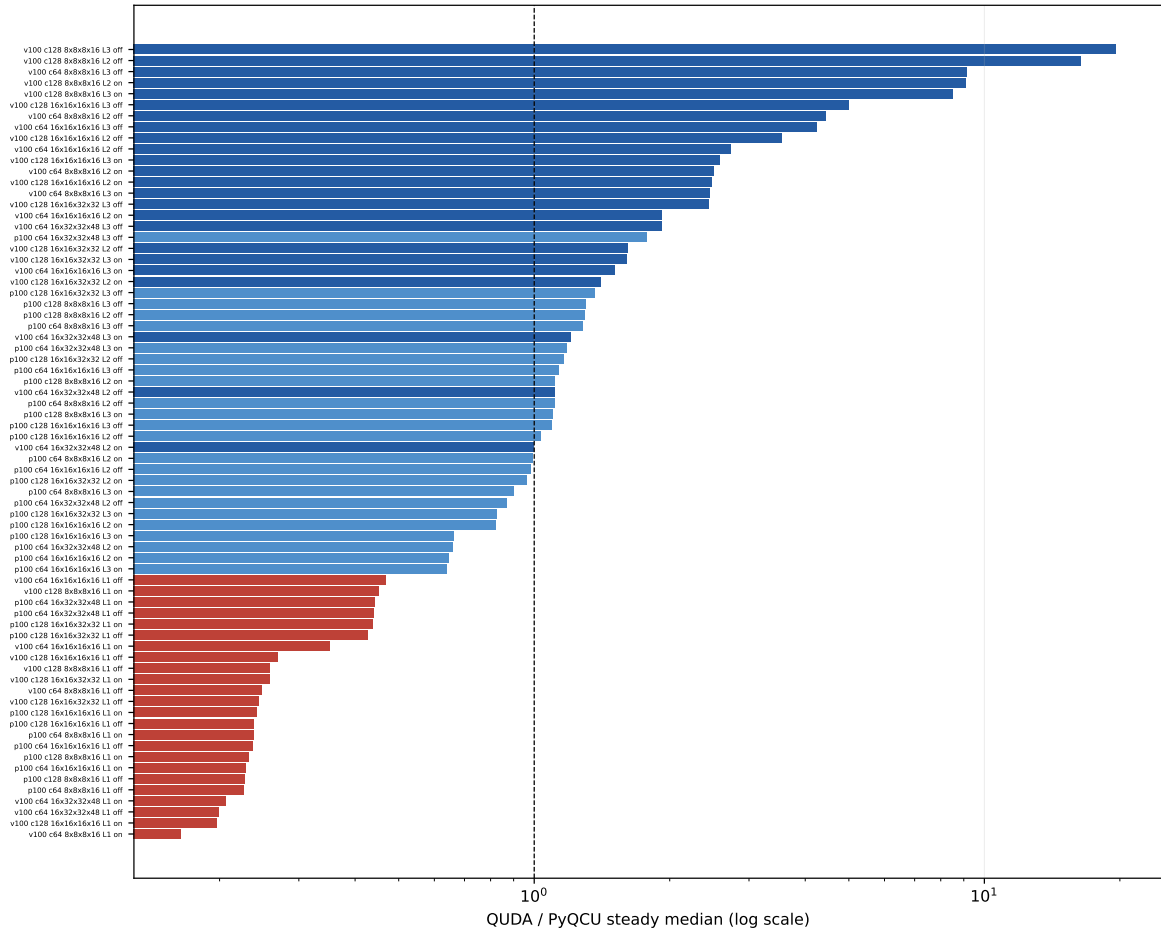


图 1: 按设备与精度统计的 QUDA/PyQCU steady 中位数比值。

6.2 逐层耗时

trace-on 数据表明, PyQCU 在双 P100 大格的主要成本仍在最粗层 BiCGStab 和 halo/归约延迟。以双 P100 c64、MG-2 trace-on 为例, PyQCU level-1 coarse solver 约 79.66s (整次 phase 的嵌套计时), 最细层 pre/post smoother 合计约 3.80s; QUDA 的同 phase coarse solver 约 28.59s。这说明后续第一优先级仍是粗层流水线, 而不是最细层 hopping。

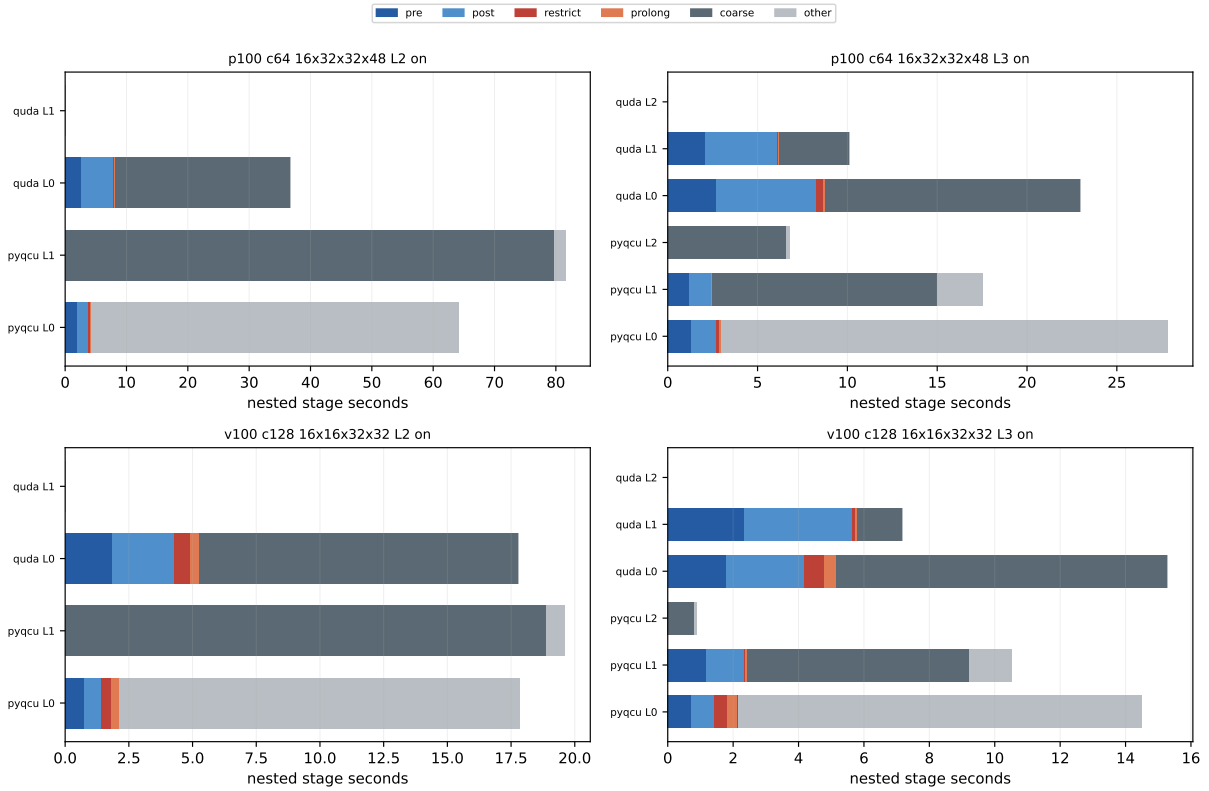


图 2: 逐层阶段时间分解 (由最终 combined JSON 生成)。



图 3: 逐层迭代计数; 最细层为 outer FGMRES/GCR, 粗层计数独立记录。

6.3 残差

所有 144 个 side case 的 true residual 均低于对应 gate。trace-on 残差序列从 combined JSON 直接提取, 不把 Arnoldi 估计值当作 true residual。图 4 展示各设备、精度、lattice、MG-2/3 的 trace-on 残差曲线。

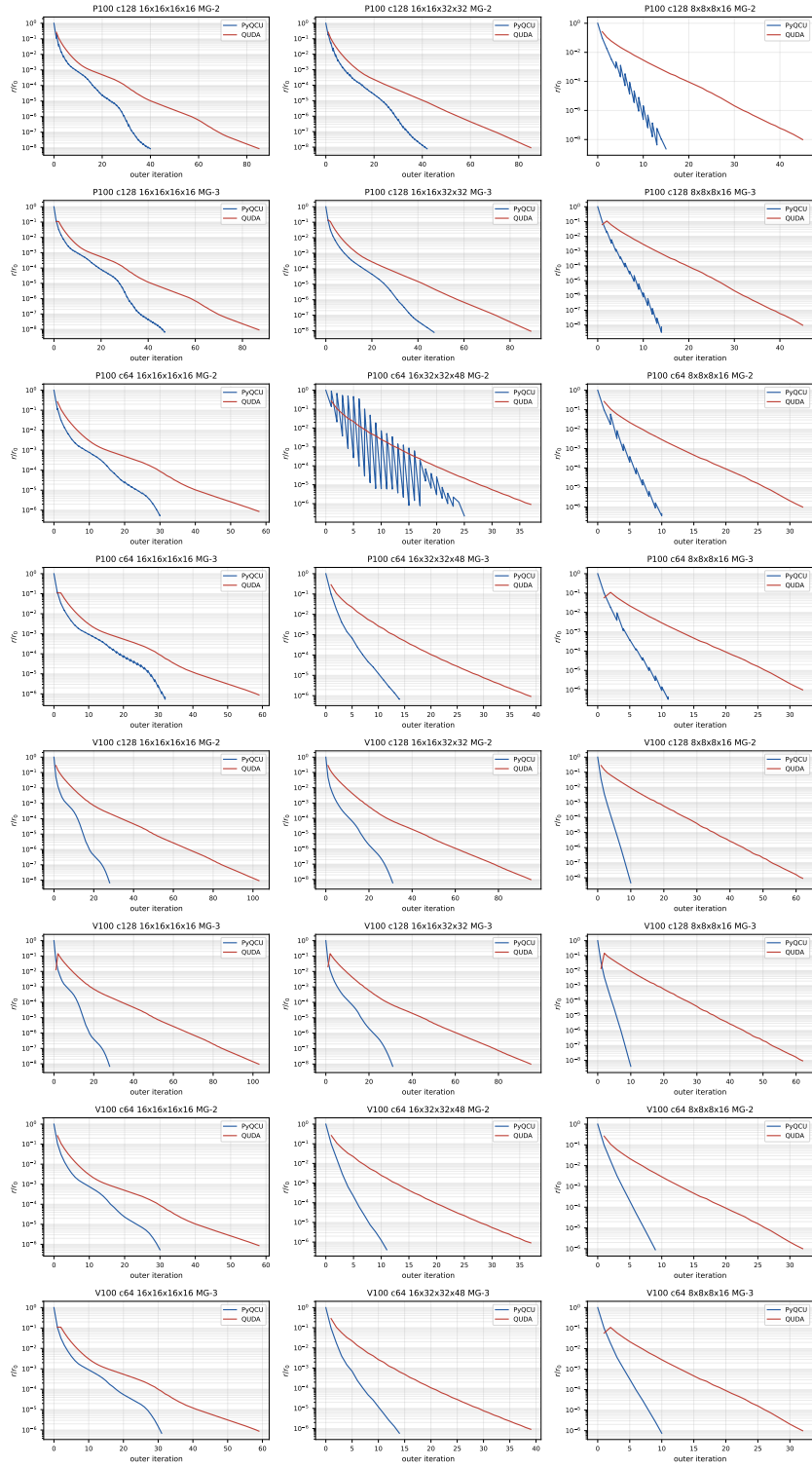


图 4: 最细层 outer residual 的迭代曲线; MG-2/3、trace-on。

No trace-on finest-level residual curves
 Required: sides.<side>.steady[].residual_curve or native trace

图 5: 各单元最细层最终真残差与 gate 的对照。

表 5: 最终验证门禁

检查	结果
<code>validate_document</code>	72/72 个 combined 文档通过
两侧 config hash	72/72 完全一致
input bundle hash	72/72 完全一致
MG level timing conservation	全部通过, 允许的 trace 归约钳零未掩盖失败
分布式 HDF5 round-trip	gauge/source/null $max_abs = 0$
CUDA/MPI 长期测试	P100 与 V100 全矩阵均完成, QUDA ghost mapped-memory 失败以重试记录, 不改算法协议
测试套件	<code>pyqcu/testing/qcu/strict/quda_comparison/test_bench_strict_protocol.py</code> 与 <code>pyqcu/testing/qcu/strict/quda_comparison/test_bench_mg_matrix_full.py</code> 全部通过 (最终回归 89 passed)

7 正确性与工程验证

8 遗留问题与适用边界

1. 双 P100 大格 c64 MG-2 仍为 0.867 $\times$ ；粗层 BiCGStab 的 host-visible reduction 与 halo 延迟没有完全消失；
2. V100 的优势包含设备代际差、单 rank 路径和不同 nu 设置，不能外推为任意 GPU/GPU 图上的普遍结论；
3. BiCGstab reference 在全部 24 个 L1 对照中慢于 QUDA，若用户关注无 MG 路径，应单独优化该 solver；
4. trace-on 的 stage 时间包含 trace 同步开销，性能比较以 trace-off 的 steady median 为准；
5. QUDA 在部分 large c128 运行中出现 `cudaHostRegister` 56 MB mapped-memory 瞬时失败；同一配置重试后成功，记录为环境级 ghost-buffer 注册问题，未修改 QUDA 数值算法；
6. 分布式读取已使用 rank-sliced h5py MPI-IO；单 rank 路径仍是完整读取，这是为了保持单卡语义和既有 cache 格式；
7. 66/72 等单元在不同 lattice/level 上存在显著方差；报告以逐单元 JSON 和 CSV 为准，不用单一平均值替代全部结果。

9 复现与产物

```
# Build final sm60+sm70 library
QCU_CUDA_ARCHITECTURES='70-real;70-virtual;60-real;60-virtual' \
  bash ./build.sh

# P100 final PyQCU side (example: c64 large)
source pyqcu/testing/qcu/strict/quda_comparison/p100_env.sh
python -B pyqcu/testing/qcu/strict/quda_comparison/bench_mg_matrix_full.py \
  --output-dir data/mg_matrix_final_p100 \
  --cache-root data/mg_matrix_full_20260921_aligned/cache \
  --only-side pyqcu --only-device p100 --execute

# V100 final PyQCU side (c128 uses nu=2; c64 uses nu=1)
CUDA_VISIBLE_DEVICES=0 python -B \
  pyqcu/testing/qcu/strict/quda_comparison/bench_mg_matrix_full.py \
  --output-dir data/mg_matrix_final_v100 \
  --cache-root data/mg_matrix_full_20260921_aligned/cache \
  --only-side pyqcu --only-device v100 --extra-args '--nu 2' \
  --execute

# Full report tables/figures
```

```
python -B pyqcu/testing/qcu/strict/quda_comparison/build_mg_report.py \
--input data/mg_matrix_final_combined/units/*.json \
--outdir data/mg_matrix_final_report \
--tex docs/report_multigrid_final_20260921.tex
python -B pyqcu/testing/qcu/strict/quda_comparison/plot_mg_residual_curves.py
```

关键产物:

- 最终 combined: data/mg_matrix_final_combined/units/;
- 性能统计: logs/data/mg_matrix_final_analysis_full.json;
- 独立校验: logs/data/mg_matrix_final_validation_full.json;
- 逐单元 CSV: data/mg_matrix_final_report/mg_matrix.csv;
- 逐层 CSV: data/mg_matrix_final_report/mg_levels.csv;
- 图: docs/data/mg_matrix_final_report/mg_speedup_units.pdf、docs/data/mg_matrix_final_report/mg_stage_breakdown.pdf、docs/data/mg_matrix_final_report/mg_level_iterations.pdf、docs/data/mg_matrix_final_report/mg_finetest_residual.pdf、docs/data/mg_matrix_final_report/mg_residual_curves.pdf。

10 结论

本轮已经完成非法内存访问 debug、真正的多 rank 输入读取、halo slot/cache 优化, 以及 144 side case 的完整测试。最终事实可以概括为:

- 正确性: 72/72 个对照全部 fair, 144 side case 真残差门禁全过;
- V100: MG-2/3 23/24 胜, 中位数 2.492 $\times$;
- 双 P100: MG-2/3 13/24 胜, 中位数 1.063 $\times$; 大格 MG-2 仍需继续优化;
- BiCGstab reference: 0/24 胜, 明确保留为未解决短板;
- 两个“降同步”粗层实验在 large c64 数值失稳后已回退, 最终版本不包含未经稳定性验证的性能路径。

11 2026-09-27 后续状态

本最终矩阵的正式 Cartesian 协议实际使用 coarse-tol=0.003; 此前快速迭代中的 0.3 仅用于小时级 A/B, 不与本表混用。后续 Strict Hopper 非阻塞 overlap、complex128 安全回退、reference 1 cold/2 warmup/5 steady 修复及最终 c03 对照见 docs/report_multigrid_optimized_20260927.tex。因此本文的性能表保留为 2026-09-21 严格容差矩阵快照, 后续报告给出新的优化版本, 不覆盖原始结果。